

1. COVER

Project: Maui Surf House

Name: Lexter Cortez

Island: Maui

Visitor Segment: Australians coming to Maui to surf

Version: v3 – May 1, 2026

GitHub:

<https://github.com/cortezlb-coder/ics385spring2026/tree/main/week15/hw15c/term-project>

Live URL: [Render URL]

2. PROBLEM STATEMENT & TARGET USER

Problem Statement

Australian surfers visiting Maui need a simple way to view surf-friendly lodging, amenities, tourism information, and current weather without searching across multiple websites. Maui Surf House combines this information into one application.

Target User

The primary user is an Australian surfer visiting Maui who wants quick, easy-to-read property and travel information. The secondary user is the property admin who needs secure access to update the property listing.

3. FUNCTIONAL REQUIREMENTS

- R1 - Display Maui Surf House property information and at least 3 surf-friendly amenities
- R2 - Display Maui tourism statistics and current weather using 3 charts
- R3 - Support local email/password authentication
- R4 - [NEW v3] Support Google OAuth 2.0 authentication
- R5 - Protect admin pages and property-editing routes
- R6 - Allow authenticated admins to update property information in MongoDB
- R7 - Allow authenticated users to log out and end their session

4. TECHNICAL ARCHITECTURE

Stack

- Frontend: React
- Backend: Node.js and Express
- Database: MongoDB and Mongoose
- Authentication: Passport.js Local and Google OAuth 2.0
- Sessions: express-session and connect-mongo
- Charts: Chart.js
- Weather: OpenWeatherMap API

Request Flow

React Frontend → Node.js/Express → MongoDB / External APIs

React requests property and authentication data from Express. Express uses Mongoose to read and update MongoDB. Chart.js displays Maui tourism data and OpenWeatherMap supplies current weather information.

Property Schema

- name: String
- island: String
- type: String
- description: String
- amenities: String[]
- targetSegment: String
- imageURL: String
- reviews: Review[]

Review Schema

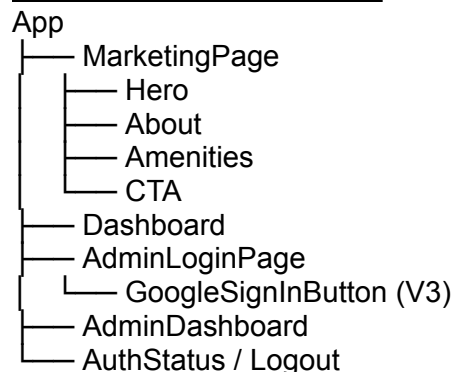
- guestName: String
- rating: Number, 1-5
- comment: String
- date: Date

Component Data

- Hero: propertyName String, island String, imageURL String, tagline String
- About: description String, location String, targetSegment String
- Amenities: amenities String[]
- CTA: buttonLabel String, buttonHref String
- Dashboard: property Property, tourismSeries Array, weather Object, weatherStatus String
- AdminLoginPage: email String, password String, errorMessage String
- GoogleSignInButton: buttonLabel String, authURL String
- AdminDashboard: property Property, editFields Object, saveStatus String
- AuthStatus: currentUser Object/null, sessionState String

The dashboard displays three charts: Visitor Arrivals, Hotel Occupancy, and Weather Forecast.

COMPONENT HIERARCHY



5. AUTHENTICATION & SECURITY

Authentication

The application supports local authentication using email and a bcrypt-hashed password and Google OAuth 2.0 using passport-google-oauth20. Local login provides a standard option while Google Sign-In provides a faster alternative.

User Data Model

- email: String, required, unique
- displayName: String
- passwordHash: String, optional for Google users
- googleId: String, unique and indexed
- provider: String, local or google
- role: String
- createdAt: Date

Account Linking

Google authentication first searches for an existing googleId, then a matching verified email. If neither exists, a new user is created in MongoDB.

Session Management

Sessions use express-session with connect-mongo and expire after 14 days. Cookies use httpOnly, sameSite "lax", and secure cookies in production.

Secret Handling

Secrets are stored in a git-ignored .env file and .env.example contains only placeholder values. Environment variables include MONGO_URI, SESSION_SECRET, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, OPENWEATHER_API_KEY, NODE_ENV, and PORT

Protected Routes

- GET /admin/dashboard - authenticated admin only
 - PUT /admin/properties/:id - authenticated admin only
 - POST /auth/logout - authenticated users only
- Unauthenticated users requesting the admin dashboard are redirected to /login. Unauthorized API requests receive HTTP 401 or 403.

Security Threats

- A01 Broken Access Control - authentication and role checks protect admin routes
- A02 Cryptographic Failures - bcrypt protects passwords and .env protects secrets
- A07 Identification and Authentication Failures - Passport.js and secure sessions protect login

6. ACCEPTANCE CRITERIA & TEST SCRIPTS

AC-1 Marketing Page

GIVEN a visitor opens Maui Surf House / WHEN the page loads / THEN the property name, Maui location, image, description, and at least 3 surf-related amenities are displayed

AC-2 Visitor Dashboard

GIVEN a visitor opens the dashboard / WHEN it loads / THEN Visitor Arrivals, Hotel Occupancy, and Weather Forecast charts display non-empty data

AC-3 Local Login

GIVEN a registered admin has valid credentials / WHEN the login form is submitted / THEN the admin is authenticated, a session is created, and the protected dashboard can be accessed

AC-4 Google Login

GIVEN a user has a Google account / WHEN Google authentication is completed / THEN the user is authenticated and the account is created or linked with a Google ID

AC-5 Protected Dashboard

GIVEN a visitor is not authenticated / WHEN the admin dashboard is requested / THEN the visitor is redirected to /login and no admin data is shown

AC-6 Property Update

GIVEN an admin is authenticated / WHEN property information is changed and saved / THEN the update is stored in MongoDB and remains after refresh

AC-7 Logout

GIVEN a user is authenticated / WHEN logout is selected / THEN the session ends and protected routes can no longer be accessed

AC-8 Secret Hygiene

GIVEN the GitHub repository / WHEN it is inspected / THEN .env is absent, .env.example is present, and no private API keys are committed

Automated Tests

- AC-3: valid local login creates an authenticated session
- AC-4: Google OAuth route redirects the user to Google authentication
- AC-5: unauthenticated users cannot access the admin dashboard
- AC-6: authenticated admins can update property information
- AC-7: logout ends the session and blocks protected routes

Test Output

[Insert screenshot of passing Jest + Supertest npm test results]

7. AI ATTRIBUTION

ChatGPT was used only to edit grammar, wording, organization, and formatting. Project requirements, design decisions, and acceptance criteria were created and reviewed by the student.